

# Práctica 4: Divide y Vencerás

Fundamentos de Análisis de Algoritmos

Autor: José Luis Salvador Martín

Fecha: Abril 2025

## Índice

- [Introducción](#)
- [Actividad 1: Análisis Teórico](#)
- [Actividad 2: Implementación y Análisis Empírico](#)
- [Actividad 3: Comparativa recursivo vs iterativo](#)
- [Actividad 4: Merge Sort y optimizaciones](#)
- [Actividad 5: Aplicación práctica](#)
- [Conclusión](#)

## Introducción

En esta práctica se analizan algoritmos de búsqueda y ordenación basados en la técnica "divide y vencerás". Se implementarán y evaluarán distintas variantes de búsqueda binaria, así como optimizaciones de Merge Sort, combinando análisis teórico con estudios empíricos para valorar su eficiencia.

## Actividad 1: Análisis Teórico

### Costes de las variantes de búsqueda binaria

#### Variante 1: Estándar

- **Mejor caso:** Ocurre cuando el elemento se encuentra exactamente en el medio del array.

Debido a esto, las operaciones que se realizarán serán mínimas (7 OE)

Por tanto, al no existir recurrencia ni bucles, estaríamos ante una complejidad Big O de  $O(1)$ , la cual es la mejor posible ya que es constante.

```
1 int busquedaBinariaI(const vector<int>& A, int izq, int der, int x) {
2     // T(n): Tiempo de ejecución para un array de tamaño n
3     if (izq > der) // 1OE -> O(1)
4         return -1;
5     int medio = (izq + der) / 2; // Cálculo del índice medio: 3 OE -> O(1)
6     if (A[medio] == x) // Comparación: 2 OE -> O(1)
7         return medio; // Elemento encontrado: 1 OE -> O(1)
8     else if (A[medio] > x)
```

$O(1)$

- **Peor caso:** El peor de los caso lo encontraremos cuando se tengan que realizar el mayor numero de iteraciones.

Esto lo asegura el caso en el cual el elemento no se encuentra en el array.

Si vamos iterando para buscar el elemento, nos damos cuenta que, cuando avanzamos en profundidad en el árbol de decisiones que se forma, al llegar a la  $k$ -ésima iteración, obtenemos  $k = \frac{n}{2^i}$ .

Por tanto, si conocemos que la sublista que nos queda es de tamaño 1, obtenemos:

```
1 int busquedaBinariaI(const vector<int>& A, int izq, int der, int x) {
2     // T(n): Tiempo de ejecución para un array de tamaño n
3     if (izq > der) // 1OE -> O(1)
4         return -1;
5     int medio = (izq + der) / 2; // Cálculo del índice medio: 3 OE -> O(1)
6     if (A[medio] == x) // Comparación: 2 OE -> O(1)
7         return medio;
8     else if (A[medio] > x) // Comparación: 2 OE -> O(1)
9         return busquedaBinariaI(A, izq, medio - 1, x); // Llamada recursiva
10    else
11        return busquedaBinariaI(A, medio + 1, der, x); // Llamada recursiva
12 }
```

$$|L| = 1 \rightarrow 2^i = n \rightarrow \log_2(2^k) = \log_2(n) \rightarrow k \cdot \log_2(2)$$

$$O(\log(n))$$

- **Caso medio:** La probabilidad de que el elemento se encuentre en cualquiera de las posiciones es  $\frac{1}{n}$ , por tanto:

$$\begin{aligned}
 T(n) &= \sum_{i=1}^n P(i) \cdot C(i) \\
 &= \sum_{i=1}^n \frac{1}{n} \cdot C(i) \\
 &= \frac{1}{n} \sum_{i=1}^n C(i) \\
 &\approx \frac{1}{n} \cdot n \cdot \log_2(n) \\
 &= \log_2(n) \\
 \Rightarrow T(n) &\in O(\log_2(n)) \subseteq O(\log n)
 \end{aligned}$$

## Variante 2: Primera ocurrencia

- **Mejor caso:** Ocurre cuando el elemento se encuentra exactamente en el medio del array y este es la primera ocurrencia, es decir, que es la ocurrencia del número más a la izquierda.

Debido a esto, las operaciones que se realizarán serán mínimas, pero constantes, variando únicamente la condición segunda.

Por tanto, al no existir recurrencia ni bucles, estaríamos ante una complejidad Big O de  $O(1)$ , la cual es la mejor posible ya que es constante.

```

1 // Variante 2: Búsqueda binaria con índice de primera ocurrencia
2 int busquedaBinaria2(const vector<int>& A, int izq, int der, int x) {
3     // T(n): Tiempo de ejecución para un array de tamaño n
4     if (izq > der) // 1OE -> O(1)
5         return -1;
6     int medio = (izq + der) / 2; // Cálculo del índice medio: 3 OE -> O(1)
7     if (A[medio] == x && (medio == izq || A[medio - 1] != x)) // Comparación: 8 OE -> O(1)
8         return medio; // Comparación: 2 OE -> O(1)
9     else if (A[medio] >= x)
10         return busquedaBinaria2(A, izq, medio - 1, x);
11     else
12         return busquedaBinaria2(A, medio + 1, der, x);
13 }

```

$O(1)$

- **Peor caso:** Igual que en la búsqueda binaria sin modificaciones, el peor de los casos lo encontraremos cuando se tengan que realizar el mayor número de iteraciones.

Esto lo asegura el caso en el cual el elemento no se encuentra en el array.

Si vamos iterando para buscar el elemento, nos damos cuenta que, cuando avanzamos en profundidad en el árbol de decisiones que se forma, al llegar a la  $k$ -ésima iteración, obtenemos  $k = \frac{n}{2^i}$ .

Por tanto, si conocemos que la sublista que nos queda es de tamaño 1, obtenemos:

```

1 // Variante 2: Búsqueda binaria con índice de primera ocurrencia
2 int busquedaBinaria2(const vector<int>& A, int izq, int der, int x) {
3     // T(n): Tiempo de ejecución para un array de tamaño n
4     if (izq > der) // 1OE -> O(1)
5         return -1;
6     int medio = (izq + der) / 2; // Cálculo del índice medio: 3 OE -> O(1)
7     if (A[medio] == x && (medio == izq || A[medio - 1] != x)) // Comparación: 8 OE -> O(1)
8         return medio; // Comparación: 2 OE -> O(1)
9     else if (A[medio] >= x) // Comparación: 2 OE -> O(1)
10         return busquedaBinaria2(A, izq, medio - 1, x);
11     else
12         return busquedaBinaria2(A, medio + 1, der, x);
13 }

```

$$|L| = 1 \rightarrow 2^i = n \rightarrow \log_2(2^k) = \log_2(n) \rightarrow k \cdot \log_2(2)$$

$O(\log(n))$

- **Caso medio:** La probabilidad de que el elemento se encuentre en cualquiera de las posiciones es  $\frac{1}{n}$ , por tanto:

$$\begin{aligned}
 T(n) &= \sum_{i=1}^n P(i) \cdot C(i) \\
 &= \sum_{i=1}^n \frac{1}{n} \cdot C(i) \\
 &= \frac{1}{n} \sum_{i=1}^n C(i) \\
 &\approx \frac{1}{n} \cdot n \cdot \log_2(n) \\
 &= \log_2(n) \\
 \Rightarrow T(n) &\in O(\log_2(n)) \subseteq O(\log n)
 \end{aligned}$$

### Variante 3: Interpolación

- **Mejor caso:** Ocurre cuando el elemento se encuentra exactamente en el medio del array y este es la primera ocurrencia, es decir, que es la ocurrencia del número más a la izquierda.

Debido a esto, las operaciones que se realizarán serán 23 OE, pero siempre manteniéndose constantes.

Por tanto, al no existir recurrencia ni bucles, estaríamos ante una complejidad Big O de  $O(1)$ , la cual es la mejor posible ya que es constante.

```
1 // Variante 3: Búsqueda binaria con interpolación
2 int busquedaBinariaInterpolacion(const vector<int>& A, int izq, int der, int x) {
3     if (izq > der || x < A[izq] || x > A[der]) // 7 OE -> O(1)
4         return -1;
5     int pos = izq + ((x - A[izq]) * (der - izq)) / (A[der] - A[izq]); //10 OE -> O(1)
6     if (pos < izq || pos > der) // 3 OE -> O(1)
7         pos = (izq + der) / 2;
8     if (A[pos] == x) // 2 OE -> O(1)
9         return pos; // 1 OE -> O(1)
10    else if (A[pos] > x)
11        return busquedaBinariaInterpolacion(A, izq, pos - 1, x);
12    else
13        return busquedaBinariaInterpolacion(A, pos + 1, der, x);
14 }
```

$O(1)$

- **Peor caso:** A diferencia de la búsqueda binaria clásica, el peor de los casos en la búsqueda por interpolación no ocurre necesariamente cuando el valor no está, sino cuando el valor se predice mal debido a una distribución muy desigual de los datos.

Esto ocurre, por ejemplo, cuando los elementos del array crecen de forma no lineal (por ejemplo: `[1, 2, 3, 1000, 1001, 1002]`).

En estos casos, la fórmula de interpolación puede llevarnos a un índice muy alejado del real, recortando muy poco en cada paso.

Si este error de predicción ocurre repetidamente, el algoritmo puede llegar a hacer hasta  $n$  comparaciones, una por cada elemento, como en la búsqueda lineal.

```
1 // Variante 3: Búsqueda binaria con interpolación
2 int busquedaBinariaInterpolacion(const vector<int>& A, int izq, int der, int x) {
3     if (izq > der || x < A[izq] || x > A[der]) // 7 OE -> O(1)
4         return -1;
5     int pos = izq + ((x - A[izq]) * (der - izq)) / (A[der] - A[izq]); //10 OE -> O(1)
6     if (pos < izq || pos > der) // 3 OE -> O(1)
7         pos = (izq + der) / 2;
8     if (A[pos] == x) // 2 OE -> O(1)
9         return pos;
10    else if (A[pos] > x) // 2 OE -> O(1)
11        return busquedaBinariaInterpolacion(A, izq, pos - 1, x);
12    else
13        return busquedaBinariaInterpolacion(A, pos + 1, der, x);
14 }
```

$$\begin{aligned} T(n) &= T(n-1) + O(1) \\ &= T(n-2) + O(1) + O(1) \\ &\dots \\ &= T(1) + (n-1) \cdot O(1) \\ &= O(n) \end{aligned}$$

$O(n)$

- **Caso medio:** Cuando el array está ordenado y los elementos están uniformemente distribuidos, la fórmula de interpolación permite predecir con bastante precisión la posición del valor buscado.

Esto hace que el algoritmo reduzca el espacio de búsqueda mucho más agresivamente que en la binaria (el caso 1 y 2), ya que no divide por la mitad, sino que salta directamente cerca del valor esperado.

Este comportamiento solo se cumple si los datos están bien distribuidos. En este caso, la búsqueda por interpolación puede ser más rápida que la binaria clásica, cuyo caso medio es  $O(\log n)$ .

$$\begin{aligned} T(n) &= T(\log n) + O(1) \\ &= T(\log \log n) + O(1) \\ &\dots \\ &\in O(\log \log n) \\ &O(\log(\log(n))) \end{aligned}$$

### Comparación BÚSQUEDABINARIA2 vs Interpolación

- Cuándo es más eficiente la interpolación
  - Como ya hemos mencionado anteriormente, la interpolación va mejor cuando los datos están ordenados de manera completamente uniforme, ya que en ese tipo de casos, acertará casi de manera inmediata la posición exacta mediante la fórmula, teniendo, en el caso medio, una complejidad de  $O(\log(\log(n)))$  vs la  $O(\log(n))$  de BUSQ2.

- Casos donde puede rendir peor
  - En los casos donde la distribución no sea uniforme (peor caso para la interpolación), obtendremos una complejidad lineal de  $O(n)$ , mientras que en el clásico modificado (BÚSQUEDABINARIA2) obtendremos como máximo una complejidad de  $O(\log(n))$
- Consideraciones numéricas
  - A pesar de que si lo que tenemos es un array “ideal” no habría problemas, en casos reales, al tener divisiones e.t.c. podemos tener problemas de redondeo.
  - Debemos tener cuidado con el desbordamiento si  $x$  y  $A[der]$  son muy grandes en la interpolación.

## Actividad 2: Implementación y Análisis Empírico

Para asegurarme de que la función `busquedaBinaria2` devuelve correctamente la **primera aparición** de un elemento cuando existen duplicados, he realizado una serie de pruebas específicas.

En concreto, he creado un array sencillo con varios valores repetidos, por ejemplo:

[1, 2, 2, 2, 3, 4, 5]

Se ha buscado entonces el número dos, dando correctamente el índice: 1 → primera ocurrencia del dos.

Esto puede comprobarse en “pruebasEspeciales.txt” donde se ha documentado el proceso.

El código que realiza esto es:

```
vector<int> repetidos = {1, 2, 2, 2, 3, 4, 5};
int dummy = 0;
int pos = busquedaBinaria2(repetidos, 0, repetidos.size() - 1, 2, dummy);
if (pos == 1)
    archivoEspeciales << "Sí (posición " << pos << ")\n";
else
    archivoEspeciales << "No (posición " << pos << ")\n";
```

Resultados:

```
===== ESTUDIO EMPÍRICO =====
```

```
>> Array vacío
```

```
Binaria1: -1, OE: 2
```

```
Binaria2: -1, OE: 2
```

```
Interpolación: -1, OE: 8
```

```
>> Array con un solo elemento (existe)
```

```
Binaria1: 0, OE: 7
```

```
Binaria2: 0, OE: 13
```

```
Interpolación: 0, OE: 23
```

```
>> Array con un solo elemento (no existe)
```

```
Binaria1: -1, OE: 12
```

```
Binaria2: -1, OE: 18
```

```
Interpolación: -1, OE: 8
```

```
>> Repetidos - buscar 1 (principio)
```

```
Binaria1: 0, OE: 27
```

```
Binaria2: 0, OE: 45
```

```
Interpolación: 0, OE: 23
```

```
>> Repetidos - buscar 2 (repetido medio)
```

```
Binaria1: 3, OE: 7
```

```
Binaria2: 1, OE: 29 → Primera esperada: 1
```

```
Interpolación: 1, OE: 23
```

```

>> Repetidos - buscar 5 (final)
  Binaria1: 6, OE: 27
  Binaria2: 6, OE: 45
  Interpolación: 6, OE: 23

>> Repetidos - buscar 6 (no existe)
  Binaria1: -1, OE: 32
  Binaria2: -1, OE: 50
  Interpolación: -1, OE: 8

>> Uniforme - buscar 0 (principio)
  Binaria1: 0, OE: 57
  Binaria2: 0, OE: 93
  Interpolación: 0, OE: 23

>> Uniforme - buscar 98 (medio)
  Binaria1: 49, OE: 7
  Binaria2: 49, OE: 13
  Interpolación: 49, OE: 23

>> Uniforme - buscar 198 (final)
  Binaria1: 99, OE: 67
  Binaria2: 99, OE: 109
  Interpolación: 99, OE: 23

>> Uniforme - buscar 199 (no existe)
  Binaria1: -1, OE: 72
  Binaria2: -1, OE: 114
  Interpolación: -1, OE: 8

>> No uniforme - buscar 1000 (final)
  Binaria1: 7, OE: 37
  Binaria2: 7, OE: 61
  Interpolación: 7, OE: 23

>> No uniforme - buscar 3 (medio)
  Binaria1: 2, OE: 27
  Binaria2: 2, OE: 45
  Interpolación: 2, OE: 75

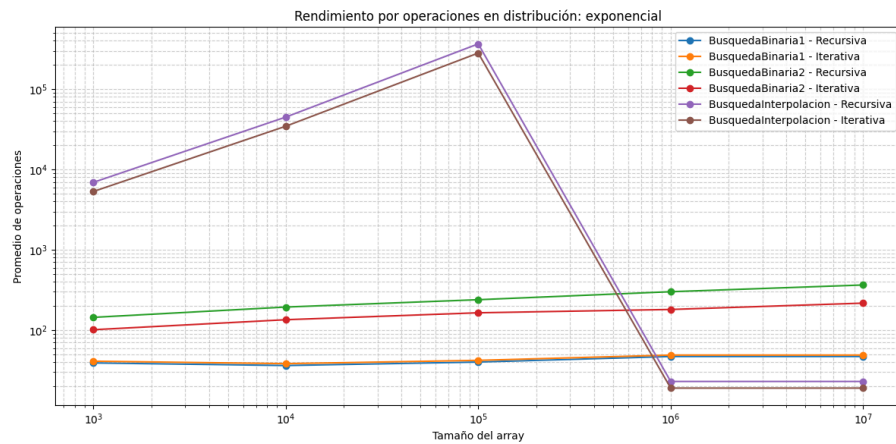
>> No uniforme - buscar 200 (no existe)
  Binaria1: -1, OE: 42
  Binaria2: -1, OE: 66
  Interpolación: -1, OE: 164

===== FIN DEL ESTUDIO =====

```

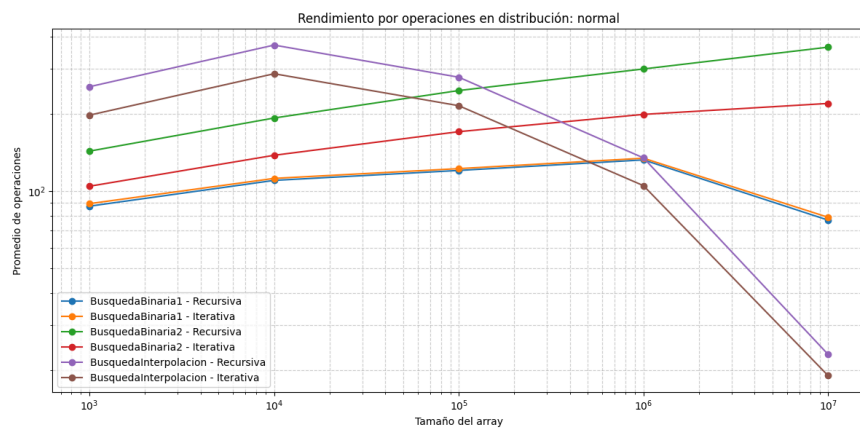
### Actividad 3: Comparativa recursivo vs iterativo

- Implementación iterativa de variantes
- Rendimiento según tamaño del array
- Comparación entre búsquedas en diferentes distribuciones
  - Distribución uniforme



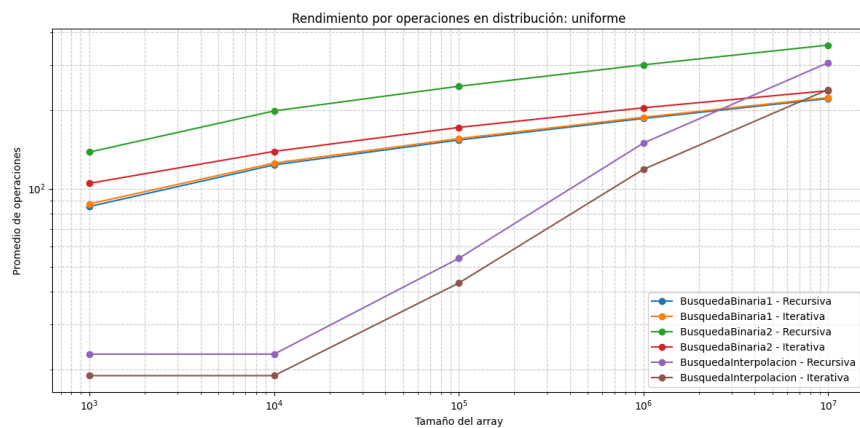
La búsqueda por interpolación ha demostrado ser la más eficiente en este caso, con un crecimiento sublogarítmico del número de operaciones, lo cual concuerda con su comportamiento teórico en el mejor de los casos ( $O(\log(\log(n)))$ ). Tanto la búsqueda binaria estándar como la de primera ocurrencia presentan un rendimiento logarítmico estable, como se espera teóricamente. La búsqueda de la primera ocurrencia resulta ligeramente más costosa debido a la necesidad de seguir buscando tras encontrar una coincidencia.

- Distribución normal



En esta distribución, la búsqueda por interpolación sufre una degradación notable en comparación con la binaria. Las estimaciones de posición que realiza el algoritmo de interpolación resultan ineficaces cuando los datos no están uniformemente distribuidos, lo que provoca un número de operaciones mucho mayor, especialmente en tamaños pequeños y medianos. Las búsquedas binarias, por su parte, mantienen un rendimiento estable.

- Distribución exponencial



La búsqueda por interpolación muestra un rendimiento muy pobre en tamaños pequeños y medios, con un número de operaciones que crece de forma casi lineal, debido a la mala distribución de los valores. No obstante, en los tamaños más grandes, su rendimiento mejora significativamente, probablemente debido a la repetición de valores en el array generado. Las búsquedas binarias mantienen nuevamente un comportamiento logarítmico predecible.

## Actividad 4: Merge Sort y optimizaciones

En la implementación clásica de Merge Sort, durante cada fusión ( `merge` ), se crean dos subarrays temporales para almacenar las mitades a combinar. Esta estrategia, aunque correcta, implica realizar un número elevado de asignaciones y accesos a memoria, especialmente si el tamaño del array es grande, lo cual puede impactar negativamente en el rendimiento práctico del algoritmo.

Para mejorar esta situación, se ha optimizado la función `merge` utilizando un único array auxiliar global, que se reserva al principio del proceso y se reutiliza en todas las fases de ordenación.

En cada llamada a `mergeOptimizado`, simplemente se copian los elementos relevantes de `A` al array auxiliar `aux` y, a partir de ahí, se realiza la fusión directamente en el array original.

Esta modificación permite:

- **Reducir el número de asignaciones de memoria dinámica**, evitando la creación continua de nuevos subarrays.
- **Minimizar los accesos redundantes a memoria**, ya que los datos se copian una sola vez.
- **Mantener el mismo coste temporal asintótico** de  $O(n \log n)$ , pero consiguiendo una mejora práctica en la eficiencia de la ordenación, especialmente para entradas de gran tamaño.

El coste espacial sigue siendo  $O(n)$  debido al uso del array auxiliar, pero el comportamiento en la práctica es más favorable tanto en velocidad como en consumo de memoria.

Resultados y comparación entre MergeSort optimizado y el normal

```
Probando tamaño n = 1000
k = 2, operaciones = 190573
k = 4, operaciones = 173075
k = 8, operaciones = 159744
k = 16, operaciones = 151375
k = 24, operaciones = 151375
k = 32, operaciones = 155173
k = 40, operaciones = 155173
k = 64, operaciones = 184592
k = 80, operaciones = 184592
k = 100, operaciones = 184592
⇒ Mejor k para n = 1000 es 16 (151375 operaciones)
-----
→ MergeSort normal (hibrido) con k=16: 151375 operaciones
→ MergeSort optimizado con k=16: 138488 operaciones
=====
Probando tamaño n = 5000
k = 2, operaciones = 1195190
k = 4, operaciones = 1135761
k = 8, operaciones = 1062183
k = 16, operaciones = 1002202
k = 24, operaciones = 978234
k = 32, operaciones = 978234
k = 40, operaciones = 1028308
k = 64, operaciones = 1028308
k = 80, operaciones = 1232582
k = 100, operaciones = 1232582
⇒ Mejor k para n = 5000 es 24 (978234 operaciones)
-----
→ MergeSort normal (hibrido) con k=24: 978234 operaciones
```

→ MergeSort optimizado con k=24: 894602 operaciones

=====

Probando tamaño n = 10000

k = 2, operaciones = 2581097

k = 4, operaciones = 2462349

k = 8, operaciones = 2316172

k = 16, operaciones = 2196792

k = 24, operaciones = 2151711

k = 32, operaciones = 2151711

k = 40, operaciones = 2256953

k = 64, operaciones = 2256953

k = 80, operaciones = 2657168

k = 100, operaciones = 2657168

⇒ Mejor k para n = 10000 es 24 (2151711 operaciones)

-----

→ MergeSort normal (hibrido) con k=24: 2151711 operaciones

→ MergeSort optimizado con k=24: 1964398 operaciones

=====

Probando tamaño n = 20000

k = 2, operaciones = 5541656

k = 4, operaciones = 5304487

k = 8, operaciones = 5010707

k = 16, operaciones = 4770108

k = 24, operaciones = 4682708

k = 32, operaciones = 4682708

k = 40, operaciones = 4884006

k = 64, operaciones = 4884006

k = 80, operaciones = 5660926

k = 100, operaciones = 5660926

⇒ Mejor k para n = 20000 es 24 (4682708 operaciones)

-----

→ MergeSort normal (hibrido) con k=24: 4682708 operaciones

→ MergeSort optimizado con k=24: 4267732 operaciones

=====

Probando tamaño n = 50000

k = 2, operaciones = 15249558

k = 4, operaciones = 14314949

k = 8, operaciones = 13559138

k = 16, operaciones = 13056902

k = 24, operaciones = 13028382

k = 32, operaciones = 13019377

k = 40, operaciones = 13019377

k = 64, operaciones = 13906530

k = 80, operaciones = 13906530

k = 100, operaciones = 16630341

⇒ Mejor k para n = 50000 es 32 (13019377 operaciones)

-----

→ MergeSort normal (hibrido) con k=32: 13019377 operaciones

→ MergeSort optimizado con k=32: 11889604 operaciones

=====

Se puede observar una clara mejora en el número de operaciones realizadas.

## Actividad 5: Aplicación práctica

- Elementos que aparecen exactamente una vez

1. Ordenamos el array **A** usando **Merge Sort** → coste  **$O(n \log n)$** .

2. **Recorremos** el array ordenado **una sola vez**:

- Si un elemento es **distinto de sus vecinos** (  $A[i-1]$  ,  $A[i+1]$  ), significa que aparece exactamente una vez.
- Comprobamos el primer y último elemento (bordes).

3. **Añadimos** los que cumplen la condición a un nuevo vector de respuesta.

4. **Devolvemos** el vector ordenado.

- Coste temporal y espacial

- Coste temporal

Nuestro algoritmo realiza dos fases principales:

- **Ordenación del array A**

Utilizamos Merge Sort (o su variante optimizada) para ordenar los elementos.

Merge Sort tiene un coste temporal de  $O(n \log n)$  en el peor caso.

- **Recorrido lineal para identificar los elementos únicos**

Una vez ordenado el array, realizamos un único recorrido de principio a fin, comparando cada elemento con sus vecinos inmediatos para comprobar si aparece exactamente una vez.

Esta fase tiene un coste  $O(n)$ .

Por tanto, el coste temporal total del algoritmo es:

$$O(n \log n) + O(n) = O(n \log n) \quad O(n \log n) + O(n) = O(n \log n)$$

$$O(n \log n) + O(n) = O(n \log n)$$

La operación dominante es la ordenación, ya que  $n \log n$  crece más rápido que  $n$  conforme aumenta el tamaño de los datos.

- Coste espacial

Respecto al uso de memoria adicional:

- **Vector auxiliar ( aux ) para Merge Sort**

Durante la ordenación mediante Merge Sort, se utiliza un array auxiliar del mismo tamaño que el original para facilitar la fusión de subarrays.

Este vector ocupa un espacio adicional de  $O(n)$ .

- **Vector de resultado ( resultado )**

El vector donde almacenamos los elementos que aparecen exactamente una vez puede, en el peor caso, contener hasta  $n$  elementos (cuando todos los elementos del array son únicos).

Por tanto, también requiere un espacio de  $O(n)$ .

- **Espacio para variables locales y constantes**

El resto de variables utilizadas (índices, contadores, etc.) ocupan un espacio constante, es decir,  $O(1)$ .

Así, el coste espacial total del algoritmo es:

$$O(n)$$

El espacio adicional necesario es proporcional al tamaño del array original.

- Resultados de la implementación

Array original: 4 5 4 6 7 5 8

Elementos únicos: 6 7 8

-----

Array original: 10 20 30 40 50

Elementos únicos: 10 20 30 40 50

-----

Array original: 1 1 1 2 2 3

Elementos únicos: 3

-----

Array original: 5 5 5 5 5

Elementos únicos:

-----

Array original:

Elementos únicos:

-----

Array original: 3 1 2 2 1 3 5 6

Elementos únicos: 5 6

-----

Array original: 100 200 100 300 200 400 500

Elementos únicos: 300 400 500

-----

- Generalización a elementos que aparecen  $k$  veces

Si ahora  $k$  fuera un **parámetro de entrada**, habría que adaptar **la fase de recorrido**.

En vez de mirar si un elemento es único respecto a sus vecinos, habría que **contar cuántas veces aparece cada valor**.

Pseudocódigo para  $k$  como parametro de entrada

```
Para i desde 0 hasta n-1:
    unico ← true
    Si (i > 0 y A[i] = A[i-1]) entonces
        unico ← false
    Si (i < n-1 y A[i] = A[i+1]) entonces
        unico ← false
    Si unico es true entonces
        añadir A[i] al resultado
```

Pseudocódigo original

```
i ← 0
Mientras i < n hacer:
    contar ← 1
    Mientras (i+1 < n) y (A[i] = A[i+1]) hacer:
        contar ← contar + 1
        i ← i + 1
    Si contar = k entonces
        añadir A[i] al resultado
    i ← i + 1
```

## Conclusión

Este trabajo analiza el comportamiento teórico y práctico de varios algoritmos mediante implementación (práctica) y evaluación empírica.

### Hallazgos principales:

1. **Búsqueda binaria:** La versión por interpolación alcanza  $O(\log \log n)$  solo en distribuciones uniformes, disminuyendo su eficiencia notablemente en datos exponenciales.
2. **Merge Sort optimizado:** La combinación con Insertion Sort para subarreglos pequeños, junto a la reutilización de memoria, mejora el rendimiento manteniendo  $O(n \log n)$ .
3. **Aplicaciones prácticas:** La estrategia de ordenación previa nos ha permitido resolver problemas de detección de elementos únicos y  $k$ -repetidos de manera eficiente ( $O(n \log n)$ ).

El análisis empírico complementa la teoría, dándonos factores determinantes (distribución de datos, optimizaciones de implementación) que el análisis asintótico por sí solo no podría arrojar.